

Cooperative UAV Trajectory Design and Resource Allocation in Blockchain-Enabled Secure Aerial Edge Computing Network

Peng Qin[✉], *Member, IEEE*, Min Fu, Yang Fu[✉], and Jingjing Wang[✉], *Senior Member, IEEE*

Abstract—Mobile Edge Computing (MEC) has emerged as a crucial technology for supporting computation-intensive and latency-sensitive Internet of things (IoT) applications. Meanwhile, UAVs can serve as MEC servers, providing cost-effective computation offloading services to IoT terminals with their flexible deployment capabilities, especially in areas lacking ground infrastructure. Nevertheless, the computation offloading process suffers from potential security risks, while the randomness and uncertainty of terminals' data sensing may exacerbate the queue backlogs. To tackle these challenges, a UAV-enabled secure aerial computing network that integrates MEC and blockchain is proposed, with the aim of jointly designing data sensing, offloading, and computing, together with UAV trajectory planning to maximize the long-term average data sensing rate under queuing delay and block creation delay constraints. To address the coupling between long-term constraints and short-term decisions, we apply Lyapunov optimization to decompose the original problem into three deterministic subproblems for each time slot. We then develop a multi-agent learning-based approach to collaboratively train terminal transmission power and UAV flight trajectories. Moreover, sensing rate and edge resource allocation are adaptively optimized in response to real-time data arrivals and queue backlogs. Simulation results demonstrate the superior performance of our solution, achieving over a 13.16% improvement in data sensing rate and more than a 29.47% reduction in queue delay compared to benchmark methods.

Index Terms—Air-ground integrated network, UAV trajectory planning, task offloading, blockchain, resource allocation, queue-awareness, multi-agent-learning.

I. INTRODUCTION

WITH the rapid development of Internet of Things (IoT) and 5G/6G wireless communication technologies, the proliferation of IoT devices has significantly increased, leading

to a surge in computation-intensive and latency-sensitive tasks. Deploying Mobile Edge Computing (MEC) servers at the network edge can effectively alleviate the burden on terminals for handling these tasks. However, in remote areas with limited infrastructure, such as deserts and wilderness, the insufficient coverage of Base Stations (BSs) makes it difficult to establish reliable ground-based MEC systems. Fortunately, with the advancement of UAVs, they can serve as edge servers with processing capabilities, and UAV-based MEC networks are gaining attention because of their adaptable mobility and Line-of-Sight (LoS) links. They can efficiently extend ground networks by providing communication [1], computation [2], and storage [3] functions from the air, thereby enhancing coverage, supporting high-speed data transmission, and delivering on-demand edge computing services. Additionally, UAV trajectory planning can be appropriately designed to further improve network service quality. Therefore, developing a multi-layer computing architecture based on Air-Ground Integrated Networks (AGIN) to support remote IoT applications is imperative [4].

However, due to the migration of computation tasks between terminals and UAVs, task offloading suffers from potential security risks such as terminal privacy leakage, the tampering and reversal of offloaded tasks as well as computation results [5]. Therefore, ensuring the security of task offloading process between terminals and UAVs in MEC is a pivotal issue. Fortunately, blockchain technology can record data in a decentralized, transparent, and secure manner through encrypted hash-linked data chains. Data stored on blockchain is immutable, making it possible to establish a transparent task offloading framework based on distributed consensus. This framework can ensure the traceability, immutability, and irreversibility of tasks, thereby effectively protecting the security and privacy of task offloading between terminals and UAVs. For instance, blockchain-based identity authentication ensures the authenticity and integrity of UAV identities, preventing forgery and tampering during task offloading. By implementing automated verification through smart contracts, it effectively mitigates security threats such as identity spoofing and man-in-the-middle attacks, ensuring secure task transmission and execution.

Moreover, to unleash the full promising of the aforementioned network architecture integrating MEC and AGIN with blockchain, several challenges remain to be tackled. *Firstly*, the sensing procedure of IoT terminals are featured by random

Received 4 January 2025; revised 28 April 2025; accepted 18 June 2025. Date of publication 30 June 2025; date of current version 22 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62201212, Grant 62271201, Grant 62222101, and Grant U24A20213; in part by the Natural Science Foundation of Hebei Province under Grant F2022502017; in part by Beijing Natural Science Foundation under Grant L232043 and Grant L222039; in part by the Natural Science Foundation of Zhejiang Province under Grant LMS25F010007; and in part by the Fundamental Research Funds for the Central Universities under Grant 2025MS008. The associate editor coordinating the review of this article and approving it for publication was J. Liu. (*Corresponding author: Peng Qin.*)

Peng Qin, Min Fu, and Yang Fu are with the State Key Laboratory of Alternate Electrical Power System with Renewable Energy Sources, School of Electrical and Electronic Engineering, North China Electric Power University, Beijing 102206, China (e-mail: qinpeng@ncepu.edu.cn).

Jingjing Wang is with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China, and also with Hangzhou Innovation Institute, Beihang University, Hangzhou 310051, China (e-mail: drwangjj@buaa.edu.cn).

Digital Object Identifier 10.1109/TWC.2025.3582151

and uncertain data amount, and it may lead to system overload, i.e., excessive data backlogs, when data sensing and offloading/processing are imbalanced. Therefore, the admission of sensed data should be dynamically controlled to stabilize the data backlogs, which, however, results in an intractable long-term stochastic issue. *Secondly*, the block generation and consensus substantially affect the system security, while competing for edge resources with data processing tasks. Consequently, designing efficient resource allocation strategy to well balance the computing and security performance, whilst considering resource restraints of UAV edge servers, is critical yet challenging. *Thirdly*, Deep Reinforcement Learning (DRL) combines the learning and prediction ability of Deep Learning (DL) with the decision-making advantage of Reinforcement Learning (RL), enabling rapid adaption to complex and dynamic networks [7], [8]. In spite of the benefits, the training of DRL suffers from slow convergence and inadequate exploration, especially in multi-agent scenarios. Thus, how to enhance DRL by achieving cooperation among multiple agents requires further research efforts.

In summary, our primary motivation is to address the security and privacy risks associated with task offloading by integrating blockchain technology into UAV-enabled edge computing networks, while supporting the long-term performance optimization of MEC systems through efficient resource coordination. Considering the randomness of task data arrivals and the dynamic nature of UAV trajectories, our goal is to design an online algorithm that maximizes the long-term average data sensing rate under queue delay and security constraints. The main contributions of this paper are as follows:

- A UAV-based secure aerial edge computing network is proposed, which includes multiple UAV edge servers and IoT devices to perform data sensing as well as computation offloading. Particularly, each UAV functions as both a computing node and a blockchain node, which dynamically allocates edge resource to simultaneously accomplish computation and blockchain tasks. Additionally, the UAV needs to design an appropriate trajectory to enhance the effectiveness of task offloading, reducing the terminal data backlogs. Subsequently, we formulate a long-term average sensing rate maximization problem by jointly considering terminal data sensing, transmission power, UAV trajectory planning, along with UAV resource allocation while adhering to long-term queue delay and security constraints.
- Because of the coupling between long-term constraints and short-term resource optimization, it is not trivial to solve the problem directly. To this end, we use Lyapunov optimization to deal with long-term queue delay and security constraints, and subsequently break down the original problem into three more manageable subproblems. First, terminal data sensing is optimized using convex optimization. Furthermore, we propose an AGIN-MASAC (Multi-Agent Soft Actor Critic) algorithm to jointly train the terminal transmission power and UAV trajectories, where each UAV and terminal device is treated as an agent operating collaboratively to enhance overall performance. Finally, the Dingo Optimization Algorithm (DOA) is used to address the edge resource allocation

TABLE I
KEY NOTATIONS

Notation	Description
$\theta_k(t), v_k(t)$	Flight angle and speed of UAV u_k
w_j	Horizontal coordinate of device d_j
$x_{j,k}(t)$	Distance from d_j to u_k
$p_{j,k}(t)$	Uplink transmission power of device d_j to UAV u_k
$R_{j,k}(t)$	Offloading transmission rate from device d_j to UAV u_k
$\gamma_{j,k}(t)$	Signal-to-interference and noise ratio of d_j at u_k
$I_j^k(t)$	Interference from other devices occupying the same frequency resource
$\eta_{j,k}^{LoS}, \eta_{j,k}^{NLoS}$	Additional losses caused by free-space propagation for LoS and NLoS links
$e_j(t)$	Admitted sensing tasks of device d_j
$Q_j(t)$	Queue for storing admitted sensing task data by device d_j
$W_j(t)$	Task data amount that leaves the queue of device d_j
$H_{j,k}(t)$	Task queue backlog stored in UAV u_k from the offloaded tasks of device d_j
$W_{j,k}^c(t)$	Amount of task processed by UAV u_k from device d_j
$f_{j,k}^c(t)$	CPU cycle frequency allocated by UAV u_k to device d_j
$\delta_j^Q(t)$	Task offloading queuing delay at device d_j
$\gamma_k(t)$	Participation of block processor u_k
$I_{ts}(t)$	Transaction size in time slot t
$D_j(t), D_{j,cr}(t)$	Computation task and result of device d_j
$Y_b(t)$	Size of the generated block
$\delta_b^g(t)$	Delay for generating a new block
$f_k^b(t)$	Computational capacity of u_k allocated to block generation
$\delta_k^{b,v}(t)$	Time for block verification
$\varphi(t)$	CPU cycles required for block verification
$f_k^{b,v}(t)$	Computational capacity of u_k allocated to block verification in time slot t
$\delta^B(t)$	Total delay for block processor u_k to complete transaction processing

issue, in which resources are judiciously assigned to data computing and block processing, adaptively balancing security and computing performance.

- We conducted numerous experiments and compared our method with four benchmark approaches including the MASAC without trajectory optimization algorithm (NT-MASAC), MASAC without power optimization algorithm (NP-MASAC), Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm, and Particle Swarm Optimization (PSO) algorithm for resource allocation. The results validate that our approach performs excellently in terms of long-term average data sensing rate, training convergence, and queuing delay.

For ease reference, the key notations are listed in Table I.

The article is arranged as follows. Related work is discussed in Section II. A UAV-based secure aerial computing network integrating MEC and blockchain technologies in Section III. Section IV addresses the problem of maximizing the terminals' long-term average data sensing rate. Section V provides a DRL-based solution. Finally, the simulation results are presented in Section VI, while Section VII offers the conclusions.

II. RELATED WORKS

A. Aerial Edge Computing Architecture Based on UAV

Some researches have investigated UAV-based aerial edge computing architectures and demonstrated their significant advantages over traditional ground networks [9], [10], [11], [12], [13]. Paper [11] jointly optimized UAV flight time, 3D trajectory, terminal offloading decisions, and time allocation for terminals, aiming to maximize the energy efficiency of UAV and consider the fairness of offloading. However, this

work overlooked the issue of scarce spectrum resources, where NOMA is crucial for enhancing spectrum efficiency. Paper [13] explored resource allocation in UAV networks using NOMA to tackle the challenge of optimizing network capacity. Paper [14] studied online cognitive data sensing and processing optimization in energy-harvesting-enabled IoT networks to maximize long-term sensing rates. However, the aforementioned work optimized the resource allocation without considering trajectory planning. Our work fills the gap by jointly optimizing terminal transmission power, UAV trajectory planning, and edge resource allocation.

B. Integration of Blockchain and MEC

Integrating blockchain with MEC has emerged as a compelling trend in recent years. Some studies [5], [15], [16], [17], [18] have conducted preliminary research on the application of blockchain technology in MEC networks. Paper [5] proposed an improved DPoS consensus scheme in a blockchain-integrated UAV-assisted MEC network. However, this work did not take task offloading and computation between terminals and UAVs into consideration. Paper [15] explored the integration of blockchain and VEC, and proposed a vehicle task allocation scheme. Paper [18] proposed a blockchain-enabled framework for task offloading and energy harvesting in multi-UAV-assisted IoT networks. The aforementioned studies overlooked the dynamic interactions between sensed data arrival and offloading decisions. Unlike these works, our approach explicitly incorporates terminal sensing rates as optimization variables and addresses the interplay between sensing, offloading, and computing within the MEC framework. Moreover, our use of Lyapunov optimization ensures long-term queue stability, distinguishing our work from prior blockchain-integrated systems.

C. Application of DRL in MEC

DRL improves learning efficiency and the capacity to handle large-scale networks, drawing widespread attention in academia for its application in solving complex resource optimization problems in MEC [19], [20], [21], [22], [23]. Paper [21] explored blockchain-based resource allocation in multi-UAV-enabled 5G-RAN systems, employing a MADRL framework to enhance the efficiency and security of virtualized resource allocation. However, the above work did not take into account long-term queue delay constraint addressed in this paper. Paper [22] used DRL to address the challenge of latency minimization by simultaneously optimizing UAV placement, resource allocation, and computation offloading, while taking into account both Quality of Service (QoS) and resource constraints. Paper [23] proposed a multi-agent DRL scheme to solve high-dimensional state space problem of joint terminal association, power allocation, and trajectory design. However, these studies overlooked the optimization of computational resources, which is particularly important in scenarios where edge server resources are limited.

We summarize the difference between our work and some other typical literature in Table II.

TABLE II
COMPARISON WITH RELATED WORKS

Reference	Scenario	Algorithm	Problem
[4]	Multi-UAV assisted network	MADDPG, greedy algorithm	Task offloading and resource management under queue-awareness
[14]	Energy-harvesting IoT network with MEC	Lyapunov, dynamic threshold-based control	Joint sensing rate and task processing optimization under energy constraints
[16]	UAV-enabled MEC network with blockchain integration	Block coordinate descent (BCD), SCA	Joint optimization of user association, UAV trajectory, block processor scheduling, and computation resource allocation
[18]	Multi-UAV-assisted IoT network	MADDPG, Stackelberg game	Computation offloading and energy harvesting, minimize resource price
[21]	UAV-enabled 5G network with blockchain	MADDPG, Stackelberg game	Resource allocation and pricing
This paper	Blockchain-integrated UAV-enabled MEC network	Lyapunov, MASAC, CVX, DOA	UAV trajectory planning, task offloading, blockchain, resource allocation

III. SYSTEM MODEL

As shown in Fig. 1, the UAV-enabled secure aerial edge computing network is composed of multiple IoT devices and UAV edge servers. Assume there are totally J devices and K UAVs. The sets of devices and UAVs are denoted as $D = \{d_1, \dots, d_j, \dots, d_J\}$ and $U = \{u_1, \dots, u_k, \dots, u_K\}$, respectively. The system operates in a time-slot mode, where the total optimization time is evenly divided into T time slots, each with a duration of τ , and the entire period is denoted as $\{1, \dots, t, \dots, T\}$. In each time slot, devices collect sensing data from the environment and store it in local task data queues. Assuming that the task data is indivisible, we adopt a full offloading strategy considering the terminals' limited resources [4], [14]. The MEC server is deployed on each UAV, which not only handles the computing tasks offloaded by terminals but also acts as a blockchain node, collecting and verifying offloading records to generate and add new blocks to the blockchain. The encryption and traceability features of the blockchain ensure the privacy and security of terminal devices during the computation offloading process effectively.

A. Communication Model

1) *UAV Trajectory*: To avoid additional power consumption during the UAV's ascent or descent, we assume that all UAVs maintain a fixed altitude of H .¹ Let $\theta_k(t) \in (-180^\circ, 180^\circ]$ and $v_k(t)$ indicate the flight angle and speed of UAV u_k during time slot t , respectively. Thus, the trajectory of UAV u_k can be expressed as

$$\mathbf{q}_k(t+1) = \mathbf{q}_k(t) + v_k(t)\tau[\cos \theta_k(t), \sin \theta_k(t)]^T. \quad (1)$$

The horizontal coordinate of device d_j is represented by $\mathbf{w}_j$, then the distance from d_j to u_k is calculated as

$$x_{j,k}(t) = \sqrt{\|\mathbf{q}_k(t) - \mathbf{w}_j(t)\|^2 + H^2}. \quad (2)$$

To reduce the offloading transmission delay as much as possible while ensuring the quality of service, each terminal only selects the association with the nearest UAV at each time slot. $a_{j,k}(t)$ indicates the correlation indicator variable, and

$$a_{j,k}(t) = \begin{cases} 1, & x_{j,k}(t) - \min_{k \in K} x_{j,k}(t) = 0 \\ 0, & \text{else.} \end{cases}$$

¹The UAV altitude is presumed to be a constant subject to government regulation [4], [5], [23], while our approach can be easily extended to 3D trajectory planning.

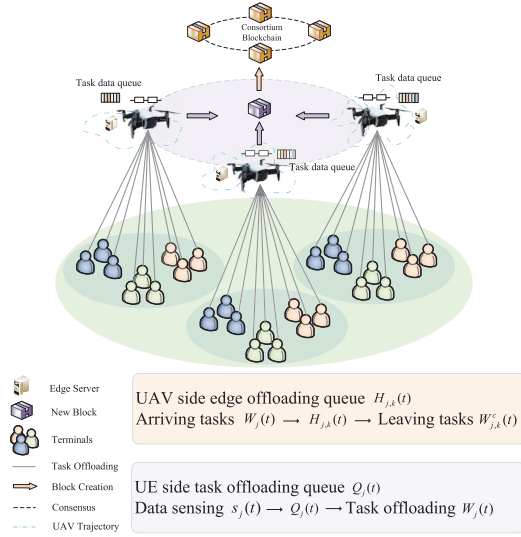


Fig. 1. The system model.

2) *Channel Model*: To further enhance spectrum efficiency, we adopt the NOMA method and the spectrum resources between UAVs are orthogonal [24]. Let B denote the bandwidth size for each cluster, and $p_{j,k}(t)$ represent the uplink transmission power of terminal d_j to UAV u_k , then the offloading transmission rate is calculated as

$$R_{j,k}(t) = B \log_2(1 + \gamma_{j,k}(t)), \quad (3)$$

where $\gamma_{j,k}(t)$ is the signal-to-interference and noise ratio of d_j at u_k , and is calculated as

$$\gamma_{j,k}(t) = \frac{p_{j,k}(t)g_{j,k}(t)}{I_j^k(t) + \sigma^2}, \quad (4)$$

where $I_j^k(t) = \sum_{i=1, i \neq j}^J a_{i,k}(t)p_{i,k}(t)g_{i,k}(t)$ represents the interference from other devices occupying the same frequency resource within the coverage area of UAV u_k , and σ^2 is the noise power. $g_{j,k}(t) = 10^{-\frac{L_{j,k}(t)}{10}}$ is defined as the channel gain, and $L_{j,k}(t)$ is the path loss from d_j to u_k , calculated by

$$L_{j,k}(t) = 20 \log_{10} \frac{4\pi f_c d_{j,k}(t)}{c} + p_{j,k}^{LoS}(t) \eta_{j,k}^{LoS} + (1 - p_{j,k}^{LoS}(t)) \eta_{j,k}^{NLoS}, \quad (5)$$

where f_c and c represent the carrier frequency and the speed of light, respectively. $\eta_{j,k}^{LoS}$ and $\eta_{j,k}^{NLoS}$ represent the additional losses caused by free-space propagation for LoS and non-line-of-sight (NLoS) links, respectively, which are determined by the environment. $p_{j,k}^{LoS}(t)$ is the LoS probability for the link from d_j to u_k , calculated by the following formula

$$p_{j,k}^{LoS}(t) = \frac{1}{1 + \rho_1 \exp \left\{ -\rho_2 \left[\tan^{-1} \left(\frac{H}{\|\mathbf{q}_k(t) - \mathbf{w}_j\|} \right) - \rho_1 \right] \right\}}, \quad (6)$$

where ρ_1 and ρ_2 are also variables determined by the environment.

B. Data Sensing and Task Queue Model

1) *Data Sensing*: The amount of sensed data for device d_j in the time slot t is defined as $s_j(t)$, which is randomly distributed within the range $[0, S_{\max}]$. To avoid system overload, the device performs admission control and only admits a portion of the sensed tasks, defined as $e_j(t)$, and the following constraint should be satisfied

$$e_j(t) \leq s_j(t). \quad (7)$$

2) *Queue Model*: Every terminal has a buffer queue for storing admitted sensing task data, denoted by $Q_j(t)$. Assuming the buffer is infinitely large and operates on a First-In-First-Out (FIFO) basis. $W_j(t)$ represents the task data amount that leaves the buffer queue of terminal d_j and is offloaded to UAV u_k , calculated as

$$W_j(t) = \sum_{k=1}^K a_{j,k}(t) \min\{Q_j(t), \tau R_{j,k}(t)\}. \quad (8)$$

Then, the queue backlog of terminal d_j in the time slot t is calculated as

$$Q_j(t+1) = \max\{Q_j(t) - W_j(t), 0\} + e_j(t). \quad (9)$$

Similarly, the task queue backlog stored in UAV u_k from the offloaded tasks of device d_j during time slot t is denoted as $H_{j,k}(t)$, and can be obtained as follows

$$H_{j,k}(t+1) = \max\{H_{j,k}(t) - W_{j,k}^c(t), 0\} + a_{j,k}(t)W_j(t), \quad (10)$$

where $W_{j,k}^c(t)$ represents the amount of task data processed by UAV u_k from device d_j . $f_{j,k}^c(t)$ denotes the CPU cycle frequency allocated by UAV u_k to device d_j . Then, $W_{j,k}^c(t)$ can be calculated by

$$W_{j,k}^c(t) = \min \left\{ H_{j,k}(t), \frac{\tau f_{j,k}^c(t)}{\lambda_j} \right\}, \quad (11)$$

where λ_j represents the computational intensity required to process the task of terminal d_j (cycles per bit).

3) *Delay Model*: Long-term queue delay encompasses both task offloading and computation queue delays. According to Little's Law, offloading queue delay is directly proportional to the queue backlog and inversely related to the average task arrival rate. Based on the previous discussion, the average task arrival rate for device d_j can be calculated as

$$\tilde{e}_j(t) = \frac{1}{t} \sum_{c=0}^{t-1} e_j(c). \quad (12)$$

Then, the task offloading queuing delay at terminal d_j can be expressed as

$$\delta_j^Q(t) = \frac{Q_j(t)}{\tilde{e}_j(t)}. \quad (13)$$

Similarly, we can obtain the task arrival rate at UAV u_k from terminal d_j and the computation queue delay as follows

$$\tilde{W}_{j,k}(t) = \frac{1}{t} \sum_{c=0}^{t-1} a_{j,k}(c)W_j(c), \quad (14)$$

$$\delta_{j,k}^H(t) = \frac{H_{j,k}(t)}{\tilde{W}_{j,k}(t)}. \quad (15)$$

4) *Energy Consumption Model*: When offloading tasks to UAV u_k during time slot t , the offloading energy consumption of terminal d_j can be calculated as

$$E_j(t) = p_{j,k}(t) \sum_{k=0}^K a_{j,k}(t) \min \left\{ \frac{Q_j(t)}{R_{j,k}(t)}, \tau \right\}. \quad (16)$$

C. Blockchain Model

UAVs acting as blockchain nodes can generate a transaction that records key information about the offloading tasks and computation results from the previous time slot. Meanwhile, a block containing the complete transaction is appended to the end of the longest confirmed chain after global consensus, and once confirmed, the block becomes irreversible. During the block consensus process, the Practical Byzantine Fault Tolerance (PBFT) protocol is used to achieve low consensus latency and a deterministic protocol.

1) *Block Generation*: To prevent excessive consumption of computational resources by blockchain nodes competing for the right to record transactions, a DPoS consensus algorithm can be used to select block processors. The algorithm employs a real-time voting mechanism to evaluate the reputation of nodes based on their past performance and the number of tokens they hold, selecting the node with the highest votes as the block processor, with all nodes eligible to participate in the voting process [16]. Note that this paper focuses on block resource allocation and does not address the selection of block processors, given that not all blockchain nodes participate in transaction recording. Therefore, in the following sections, we leverage $\gamma_k(t)$ to indicate the participation of block processor u_k , where $\gamma_k(t) = 1$ means u_k participates in the transaction recording in the time slot t , otherwise $\gamma_k(t) = 0$.

The transaction size in time slot t can be calculated as

$$I_{ts}(t) = Hash \left(\sum_{j \in J} (D_j(t) + D_{j,cr}(t)) \right) + D_{ef}(t), \quad (17)$$

where $Hash(\cdot)$ is a hash function which is used to convert the computation tasks and results into hash values for conserving UAV storage space. $D_j(t)$ and $D_{j,cr}(t)$ represent the computation task and result of terminal d_j , respectively, $D_{ef}(t)$ denotes the associated encrypted files. Meanwhile, we denote the size of the generated block as $Y_b(t)$.

For block processor u_k , the delay for generating a new block could be represented as

$$\delta_k^g(t) = \frac{I_{ts}(t)L_k(t)}{f_k^b(t)}, \quad (18)$$

where $f_k^b(t)$ represents the computational capacity of u_k allocated to block generation in time slot t , and $L_k(t)$ represents the CPU cycles required by u_k for generating each bit of block data.

2) *Consensus Process*: The consensus process involves block propagation and verification [25].

Consensus: Due to varying distances and transmission rates between different UAV block processors, we represent the

block propagation delay as the transmission delay required for the latest receipt of the block, calculated as

$$\delta_k^{b,p}(t) = \frac{Y_b(t)}{R_{k,\min}(t)}, \quad (19)$$

where $R_{k,\min}(t) = \min_{k' \in K \setminus \{k\}} \gamma_{k'}(t)R_{k,k'}(t)$, and $R_{k,k'}(t)$ represents the transmission rate from block processor u_k to block processor $u_{k'}$.

Block verification: Similarly, the time for block verification is represented as the maximum delay required to verify across all blocks, calculated as

$$\delta_k^{b,v}(t) = \max_{k' \in K \setminus \{k\}} \gamma_{k'}(t) \frac{\varphi(t)}{f_{k'}^{b,v}(t)}, \quad (20)$$

where $\varphi(t)$ denotes the CPU cycles required for block verification in time slot t , and $f_{k'}^{b,v}(t)$ represents the computational capacity of $u_{k'}$ allocated to block verification in time slot t .

Thus, the total delay for block processor u_k to complete transaction processing is given by

$$\delta^B(t) = \sum_{k=1}^K \gamma_k(t) [\delta_k^g(t) + \delta_k^{b,p}(t) + \delta_k^{b,v}(t)]. \quad (21)$$

Since block creation delay significantly impacts the security of edge computing, we will impose a security constraint on the long-term block creation delay, as follows

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \delta^B(t) \leq \delta_{\max}^B, \quad (22)$$

where $\delta_{\max}^B$ is the security upper bound for block creation delay.

IV. PROBLEM FORMULATION AND TRANSFORMATION

In this section, we propose a problem to maximize the long-term average data sensing rate for terminals in a UAV-enabled aerial secure edge computing network, subject to long-term queue and security constraints.

A. Problem Formulation

Our optimization goal is to maximize the long-term average data sensing rate by jointly optimizing UAV trajectory $\mathbf{q}(t)$, terminal transmission power $\mathbf{p}(t)$, edge resource allocation $\mathbf{f}(t)$ (includes computation $f_{j,k}^c(t)$, block generation $f_k^b(t)$, block verification $f_k^{b,v}(t)$), and task sensing rate $\mathbf{e}(t)$, while adhering to long-term queue delay and security constraints. The problem can be formulated as follows

$$\begin{aligned} \text{P0 : } & \max_{\{\mathbf{q}(t), \mathbf{p}(t), \mathbf{f}(t), \mathbf{e}(t)\}} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{j=1}^J e_j(t) \\ \text{s.t. } & C_1 : \sum_{j=1}^J f_{j,k}^e(t) + \gamma_k(t) f_k^b(t) \\ & \quad + \gamma_k(t) f_k^{b,v}(t) \leq f_{k,\max}(t), \quad \forall u_k \in U, \\ & C_2 : f_{j,k}^e(t) \geq 0, f_k^b(t) \geq 0, f_k^{b,v}(t) \geq 0, \\ & \quad \forall u_k \in U, \end{aligned}$$

$$\begin{aligned}
C_3 : 0 \leq p_{j,k}(t) \leq p_j^{\max}, \quad \forall d_j \in D, \\
C_4 : \|\mathbf{q}_k(t+1) - \mathbf{q}_k(t)\| \leq v^{\max} \tau, \\
\forall u_k \in U, \\
C_5 : \|\mathbf{q}_k(t+1) - \mathbf{q}_i(t)\| \geq \xi, \\
\forall u_k, u_i \in U, k \neq i, \\
C_6 : cs_d \leq \mathbf{q}_k(t) \leq cs_u, \quad \forall u_k \in U, \\
C_7 : e_j(t) \leq s_j(t), \quad \forall d_j \in D, \\
C_8 : \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \delta_j^Q(t) \leq \delta_{j,\max}^Q, \\
C_9 : \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \delta_{j,k}^H(t) \leq \delta_{j,k,\max}^H, \\
C_{10} : \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \delta^B(t) \leq \delta_{\max}^B. \quad (23)
\end{aligned}$$

where $C_1 - C_2$ represent the constraints on computational resource allocation for UAVs. C_3 denotes terminal transmission power constraint. C_4 is the mobility constraint with $v^{\max}$ being the maximum UAV speed. C_5 represents the anti-collision constraint to ensure UAV flight safety, where ξ indicates the minimum distance between UAVs. C_6 indicates the constraint within the UAV service area, with cs_u and cs_d representing the upper and lower bounds, respectively. C_7 denotes the constraint on the acceptance of a limited amount of sensing data. C_8 represents the task offloading queuing delay constraint. C_9 denotes the edge computing queuing delay constraint. C_{10} is the constraint on long-term security.

Problem **P0** is inherently NP-hard due to the strong coupling between long-term constraints $C_8 - C_{10}$ and short-term variables (e.g., UAV position and resource allocation in each slot). Although directly optimizing the decisions over all time slots could lead to the optimal performance, the dependency across high-dimensional variables significantly increases the solution complexity. Besides, the uncertainty of dynamic environments (e.g., future task arrival $s_j(t)$) hinders us to employ offline optimization, especially in real-time scenarios.

B. Problem Transformation

To address long-term queuing delay constraints, we introduce two new virtual queues for task offloading and edge computing, and transform C_8 and C_9 into queue stability constraints as follows

$$V_j^Q(t+1) = \left\{ V_j^Q(t) + \delta_j^Q(t+1) - \delta_{j,\max}^Q \right\}^+, \quad (24)$$

$$V_{j,k}^H(t+1) = \left\{ V_{j,k}^H(t) + \delta_{j,k}^H(t+1) - \delta_{j,k,\max}^H \right\}^+. \quad (25)$$

If the above two virtual queues are stable with respect to their average rates, then C_8 and C_9 are automatically satisfied, i.e.

$$\lim_{t \rightarrow \infty} \mathbb{E}[V_j^Q(t)]/t = 0, \quad (26)$$

$$\lim_{t \rightarrow \infty} \mathbb{E}[V_{j,k}^H(t)]/t = 0. \quad (27)$$

Additionally, introduce a virtual queue $V^B(t)$ related to C_{10} , represented as

$$V^B(t+1) = \left\{ V^B(t) + \delta^B(t) - \delta_{\max}^B \right\}^+. \quad (28)$$

The larger the value of $V^B(t)$, the more the block creation delay deviates significantly from the security requirements.

Subsequently, **P0** is transformed into

$$\begin{aligned}
\mathbf{P1} : \max_{\{\mathbf{q}(t), \mathbf{p}(t), \mathbf{f}(t), \mathbf{e}(t)\}} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{j=1}^J e_j(t) \\
\text{s.t. } C1, C2, C3, C4, C5, C6, C7, \\
C_8' : (24) \text{ is mean rate stable,} \\
C_9' : (25) \text{ is mean rate stable,} \\
C_{10}' : (28) \text{ is mean rate stable.} \quad (29)
\end{aligned}$$

The transformed problem remains NP-hard, and the interplay between short-term decisions and long-term queue delays complicates its solution. Therefore, we use Lyapunov optimization technology to transform the problem into a series of short-term, deterministic optimization subproblems that can be solved on a per-time-slot basis. We define the Lyapunov function as half the sum of the squares of all queues, given by

$$\begin{aligned}
L[\Theta(t)] = \frac{1}{2} \sum_{j=1}^J \left\{ (Q_j(t))^2 + (V_j^Q(t))^2 + \sum_{k=1}^K [(H_{j,k}(t))^2 \right. \\
\left. + (V_{j,k}^H(t))^2] \right\} + \frac{1}{2} (V^B(t))^2. \quad (30)
\end{aligned}$$

The Lyapunov drift is the expected change in the Lyapunov function between consecutive time slots, as follows

$$\Delta L[\Theta(t)] = \mathbb{E}\{L[\Theta(t+1)] - L[\Theta(t)] \mid \Theta(t)\}. \quad (31)$$

Then, the drift-plus-penalty approach is used for maintaining the queue stability while simultaneously maximizing long-term average data sensing rate. The derivation is as follows

$$\Delta_V L[\Theta(t)] = \Delta L[\Theta(t)] - \lambda \mathbb{E} \left\{ \sum_{j=1}^J e_j(t) \mid \Theta(t) \right\}, \quad (32)$$

where $\lambda > 0$, which is introduced for balancing penalty minimization and queue stability.

Theorem 1: Given the network state, the upper bound of $\Delta_V L[\Theta(t)]$ is further derived as

$$\begin{aligned}
\Delta_V L[\Theta(t)] &\leq C - \left\{ \lambda \sum_{j=1}^J \mathbb{E}\{e_j(t) \mid \Theta(t)\} \right. \\
&\quad - \sum_{j=1}^J \mathbb{E}\{Q_j(t) e_j(t) \mid \Theta(t)\} - \frac{1}{2} \sum_{j=1}^J \mathbb{E}\{[e_j(t)]^2 \mid \Theta(t)\} \\
&\quad - \sum_{j=1}^J \mathbb{E} \left\{ (t+1) V_j^Q(t) \frac{e_j(t)}{\sum_{c=0}^{t-1} e_j(c) + e_j(t)} \middle| \Theta(t) \right\} \Bigg\} \\
&\quad - \sum_{j=1}^J \mathbb{E}\{Q_j(t) W_j(t) \mid \Theta(t)\} - \frac{1}{2} \sum_{j=1}^J \mathbb{E}\{[W_j(t)]^2 \mid \Theta(t)\}
\end{aligned}$$

$$\begin{aligned}
& - \sum_{j=1}^J \mathbb{E} \left\{ (t+1) V_j^Q(t) \frac{Q_j(t) - W_j(t)}{\sum_{c=0}^{t-1} e_j(c) + e_j(t)} \middle| \Theta(t) \right\} \\
& - \sum_{j=1}^J \sum_{k=1}^K \mathbb{E} \left\{ H_{j,k}(t) a_j^k(t) W_j(t) \middle| \Theta(t) \right\} - \sum_{j=1}^J \sum_{k=1}^K \left\{ (t+1) \right. \\
& \quad \times V_{j,k}^H(t) \frac{a_{j,k}(t) W_j(t) + H_{j,k}(t)}{\sum_{c=0}^{t-1} a_{j,k}(c) W_j(c) + a_{j,k}(t) W_j(t)} \middle| \Theta(t) \left. \right\} \\
& + \sum_{j=1}^J \sum_{k=1}^K \mathbb{E} \left\{ H_{j,k}(t) \frac{\tau f_{j,k}^c(t)}{\lambda_j} \middle| \Theta(t) \right\} \\
& - \mathbb{E} \left\{ V^B(t) \delta^B(t) \middle| \Theta(t) \right\} + \sum_{j=1}^J \sum_{k=1}^K \left\{ (t+1) V_{j,k}^H(t) \right. \\
& \quad \times \frac{\tau f_{j,k}^c(t)}{\lambda_j} \left. \middle| \Theta(t) \right\} \times \frac{\tau f_{j,k}^c(t)}{\sum_{c=0}^{t-1} a_{j,k}(c) W_j(c) + a_{j,k}(t) W_j(t)} \left. \right\} \quad (33)
\end{aligned}$$

where C is a constant.

Proof: See Appendix A. $\blacksquare$

We observe the system state and apply the opportunity cost minimization approach to optimize system parameters at each time slot t . Consequently, the original problem **P1** can be reformulated as minimizing the upper bound of the expression derived in (33) under the given constraints. Based on the involved variables, **P1** is decomposed into three sequential, distributed subproblems.

1) *Data Sensing Rate Optimization (SP1):* In **SP1**, the terminal data sensing rate is set independently of other system variables. The problem can be expressed as

$$\begin{aligned}
\mathbf{SP1} : \max_{\mathbf{e}(t)} \phi_j(t) &= \lambda e_j(t) - Q_j(t) e_j(t) - \frac{1}{2} [e_j(t)]^2 \\
&\quad - (t+1) V_j^Q(t) \frac{e_j(t)}{\sum_{c=0}^{t-1} e_j(c) + e_j(t)}, \\
\text{s.t. } & C7. \quad (34)
\end{aligned}$$

2) *UAV Trajectory Planning and Terminal Transmission Power Control (SP2):* In **SP2**, terminals select the optimal transmission power while UAVs optimize their flight trajectories.

$$\begin{aligned}
\mathbf{SP2} : \max_{\mathbf{q}(t), \mathbf{p}(t)} \varphi_j(t) &= Q_j(t) W_j(t) - \frac{1}{2} [W_j(t)]^2 \\
&\quad - (t+1) V_j^Q(t) \frac{Q_j(t) - W_j(t)}{\sum_{c=0}^{t-1} e_j(c) + e_j(t)} \\
&\quad - \sum_{k=1}^K H_{j,k}(t) a_j^k(t) W_j(t) \\
&\quad - \sum_{k=1}^K (t+1) V_{j,k}^H(t)
\end{aligned}$$

$$\begin{aligned}
& \times \frac{a_{j,k}(t) W_j(t) + H_{j,k}(t)}{\sum_{c=0}^{t-1} a_{j,k}(c) W_j(c) + a_{j,k}(t) W_j(t)}, \\
\text{s.t. } & C3, C4, C5, C6. \quad (35)
\end{aligned}$$

It is still an NP-hard non-convex, programming challenge, since the UAV trajectory is coupled with the terminal transmission power. Besides, the high dynamics of the network and strict delay requirements necessitate a rapid solution. Therefore, we propose a MASAC-based approach in the next section.

3) *UAV Resource Allocation (SP3):* In **SP3**, the UAV allocates resources to both terminal tasks and blockchain processing at the time slot t .

$$\begin{aligned}
\mathbf{SP3} : \max_{\mathbf{f}(t)} \psi_k(t) &= \sum_{k=1}^K H_{j,k}(t) \frac{\tau f_{j,k}^c(t)}{\lambda_j} - V^B(t) \delta^B(t) \\
&\quad + \sum_{k=1}^K (t+1) V_{j,k}^H(t) \\
&\quad \times \frac{\tau f_{j,k}^c(t)}{\sum_{c=0}^{t-1} a_{j,k}(c) W_j(c) + a_{j,k}(t) W_j(t)} \\
\text{s.t. } & C1, C2, \\
& C10 : \frac{\tau f_{j,k}^c(t)}{\lambda_j} \leq H_{j,k}(t). \quad (36)
\end{aligned}$$

V. MASAC-BASED TRAJECTORY PLANNING AND RESOURCE ALLOCATION ALGORITHM

In this section, we propose a MASAC-based approach to address the joint UAV trajectory planning and terminal power control subproblem in aerial edge computing networks, treating UAVs and terminal devices as two distinct types of agents. Besides, we use convex optimization and heuristic algorithm to solve the terminal data sensing rate and UAV resource allocation subproblems, respectively.

A. Optimization of Data Sensing Rate

In **SP1**, we define $z_j(t) = \sum_{c=0}^{t-1} e_j(c)$, and then transform the original problem into its dual problem, as follows

$$\begin{aligned}
\mathbf{SP1} : \max_{\mathbf{e}(t)} \phi_j(t) \\
\text{s.t. } & C7, \\
& z_j(t) = \sum_{c=0}^{t-1} e_j(c). \quad (37)
\end{aligned}$$

The second derivative of the objective function is $\frac{\partial^2 \phi_j(t)}{\partial e_j(t)^2} = -1 + 2(t+1) \frac{z_j(t)}{(z_j(t) + e_j(t))^3} V_j^Q(t)$. Given that $e_j(t)$ is on the order of 10^5 , and $V_j^Q(t) < z_j(t) + e_j(t)$, so $2(t+1) \frac{z_j(t)}{(z_j(t) + e_j(t))^3} V_j^Q(t) \ll 1$. Thus, we can obtain $\frac{\partial^2 \phi_j(t)}{\partial e_j(t)^2} < 0$. Therefore, **SP1** is a convex optimization problem with linearity constraints, which can be solved using existing optimization tools, such as CVX.

²<https://pan.baidu.com/s/1SblAQDLHkjsGTImQwUP29w?pwd=nfk7>

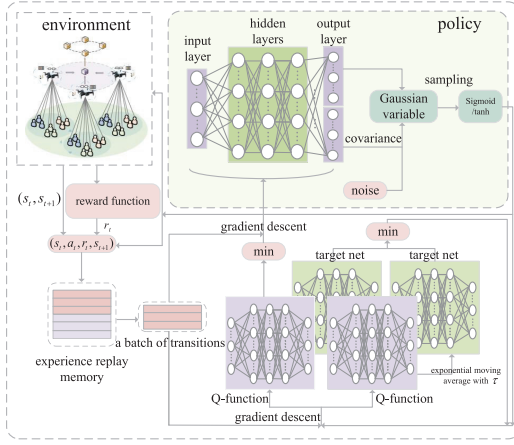


Fig. 2. The architecture of SAC.

B. UAV Trajectory Planning and Terminal Power Control

1) *The SAC Framework:* SAC is an off-policy reinforcement learning algorithm aimed at maximizing the weighted sum of cumulative rewards and policy entropy, encouraging the agent to exhibit greater randomness in action selection. By combining offline policies with maximum entropy reinforcement learning, SAC improves sampling efficiency and stabilizes convergence. The maximum entropy policy seeks to maximize the expected return and entropy simultaneously.

Fig. 2 illustrates the SAC structure, which includes a policy function (indicated by $\pi_\phi(a_t|s_t)$), two Q-functions (indicated by $Q_{\beta_1}(s_t, a_t)$ and $Q_{\beta_2}(s_t, a_t)$), two target networks (indicated by $Q_{\bar{\beta}_1}(s_t, a_t)$ and $Q_{\bar{\beta}_2}(s_t, a_t)$), and an experience replay pool, where s_t and a_t denote environment state and action, respectively. Nevertheless, SAC is a single-agent DRL algorithm that does not account for inter-agent interference. While multiple agents update their policies simultaneously, it can lead to an unstable environment and training process. Therefore, we invoke MASAC to address the inherent non-stationarity of multi-agent environments and make decisions on terminal power control and UAV trajectory planning under the interference of resource competition among terminals.

2) *Policy Function:* The policy function π_ϕ , acting as the “actor”, executes actions. Besides, the policy gathers state information from the network at each time slot.

State: The terminal’s observations include the terminal’s position, queue backlog, historical transmission power, and the position of the associated UAV. Thus, the state space is denoted as

$$s_j(t) = \{\mathbf{q}_k(t), \mathbf{w}_j(t), Q_j(t), p_{j,k}(t-1) : \forall k\}. \quad (38)$$

Additionally, the UAV’s observations include the positions of all devices and UAVs, queue backlogs, and historical decisions, represented as

$$s_{J+k}(t) = \{\{\mathbf{q}_k(t) : \forall k \in K\}, \{\mathbf{w}_j(t) : \forall j \in J\}, \theta_k(t-1), v_k(t-1), \{H_{j,k}(t) : \forall k \in K\}\}. \quad (39)$$

Action: The policy function takes actions based on the state to adjust the UAV’s flight direction angle and the transmission

power of each terminal. Specifically, for terminal, the action space is represented as

$$a_j(t) = \{p_{j,k}(t)\}. \quad (40)$$

Meanwhile, the action space of UAV is expressed as³

$$a_{J+k}(t) = \{\theta_k(t), v_k(t)\}. \quad (41)$$

The policy function is executed using a fully connected deep neural network (DNN), with parameters denoted as ϕ . During DNN training, the output layer produces two vectors of the same dimension, which serve as the mean and covariance for Gaussian random variables generation. The Gaussian random variables are sampled, and their values are constrained through an activation function, resulting in the policy function producing action $a(t)$.

However, the actions determined by the policy function may not fulfill all the constraints of **SP2**, hence we design several tricks below. For the terminal power control, the activation function of the output layer is a sigmoid function, and the output values are constrained to the desired range through a linear transformation. Specifically, denote the Gaussian random variable generated by the policy function as z , which is transformed into actual action P_{output} via

$$\begin{aligned} P_{output} &= P_{\min} + (P_{\max} - P_{\min}) \cdot \sigma(z) \\ &= P_{\max} \cdot \sigma(z). \end{aligned} \quad (42)$$

where $\sigma(\cdot)$ is the sigmoid activation function. Likewise, we use the sigmoid and tanh function to output the UAV’s speed and direction angle, respectively, followed by scaling to the feasible range. In this way, the policy function can generate actions that comply with constraints related to terminal power control and UAV trajectory planning.

Reward: Besides, we further guarantee safety distance constraint C_5 and service area constraints C_6 by adding penalties for violations. Define binary factors $\xi_k^s(t)$ and $\xi_k^c(t)$ to indicate whether UAV k flies close to another UAV and out of the service area, respectively. That is, $\xi_k^s(t) = 1$ indicates UAV k violates the safety constraint, and $\xi_k^c(t) = 1$ indicates the UAV flies out of the service area, otherwise $\xi_k^s(t), \xi_k^c(t) = 0$. Subsequently, SAC outputs the adjusted action $a(t)$ and receives a reward $r(t)$. We consider a collaborative multi-agent architecture where all agents aim to maximize a common reward. The reward design should primarily focus on minimizing the objective function of SP2. As a result, the reward can be defined as

$$r(t) = \sum_{j=1}^J \varphi_j(t) - \sum_{k=1}^K [\xi_k^s(t)r_t^s + \xi_k^c(t)r_t^c], \quad (43)$$

where r_t^s and r_t^c denote the penalties for breaching the constraints C_5 and C_6 , respectively.

3) *Experience Replay Pool:* Upon receiving $r(t)$ and the subsequent time slot’s state $s(t+1)$, SAC stores the transition tuple $(s(t), a(t), r(t), s(t+1))$ in the experience replay memory as a training experience. When the memory reaches

³If we consider 3D trajectory optimization, an additional dimension corresponding to UAV vertical direction angle should be added in (41).

capacity, newly generated transitions replace the oldest transitions. Besides, SAC periodically draws a random batch of transitions from the memory and applies gradient descent to update the neural networks of the policy function and Q-functions.

4) *Q-Functions*: The ‘critic’ uses two Q-functions, Q_{β_1} and Q_{β_2} during gradient descent, aiming at reducing the positive bias in policy improvement. Q_{β_1} and Q_{β_2} are implemented by two DNNs with parameters β_1 and β_2 , respectively. Both produce Q-values for state-action pairs, with SAC choosing the smaller of the two Q-values.

5) *Target Networks*: The DNNs for every Q-function are updated via gradient descent as well, with two target networks, $Q_{\bar{\beta}_1}$ and $Q_{\bar{\beta}_2}$, used to lessen the correlation between transitions and stabilize training. As backups for the Q-functions, the two target networks initially have the same structure and parameters as the Q-functions. Then, they adjust their parameters based on the exponential moving averages of Q_{β_1} and Q_{β_2} , along with a smoothing constant τ .

6) *Maximum Entropy Reinforcement Learning*: The objective of SAC is to maximize both the expected cumulative reward as well as the anticipated entropy of the policy. Thus, the maximum entropy objective is defined as

$$J(\pi) = \sum_{t=1}^T \mathbb{E}_{(s(t), a(t)) \sim \rho_\pi} [r(t) - \alpha \log(\pi(\cdot|s(t)))]. \quad (44)$$

where α is a smoothing coefficient that adjusts the balance between the system’s reward and the exploration of unknown actions.

SAC carries out gradient descent on the neural networks for both the Q-function and the policy function at fixed intervals. The parameters $\beta_i, i = 1, 2$ of the Q-function are refined through minimizing the soft Bellman residual

$$J_Q(\beta_i) = \mathbb{E}_{(s(t), a(t)) \sim D} \left[\frac{1}{2} \left(Q_{\beta_i}(s(t), a(t)) - \left(r(t) + \gamma \min_{z=1,2} [Q_{\bar{\beta}_z}(s(t+1), a(t+1))] - \alpha \log(\pi_\phi(a(t+1)|s(t+1))) \right) \right)^2 \right]. \quad (45)$$

where D represents the distribution of states and actions that are sampled.

The policy function parameter ϕ is updated in the following way

$$J_\pi(\phi) = \mathbb{E}_{s(t) \sim D} [\alpha \log \pi_\phi(f_\phi(\varepsilon(t); s(t))|s(t)) - \min_{z=1,2} Q_{\bar{\beta}_z}(s(t), f_\phi(\varepsilon(t); s(t)))]. \quad (46)$$

Here, the reparameterization trick addresses the policy gradient problem by expressing the policy as $a(t) = f_\phi(\varepsilon(t); s(t))$, where $\varepsilon(t)$ denotes the independent noise vector.

7) *Implementation of MASAC*: As shown in **Algorithm 1**, every agent (UAV and terminal) collects state information $s(t)$ and performs actions $a(t)$ based on observations in each time slot. After executing the action $a(t)$, a reward $r(t)$ is received and the transition $(s(t), a(t), r(t), s(t+1))$ is stored in the experience replay buffer, while the neural network parameters

Algorithm 1 MASAC-Based Algorithm for UAV Trajectory Planning and Terminal Power Control (AGIN-MASAC)

- 1: **Input**: UAV and terminal actor and critic networks, task queue-related information $\{Q_j(t), H_{j,k}(t), V_j^Q(t), V_{j,k}^H(t), \tilde{e}_j(t), \tilde{W}_{j,k}(t)\}$.
 - 2: **Output**: Optimal terminal power and UAV flight direction.
 - 3: **Initialize**: Network parameters for policy function and Q-functions: ϕ, β_1, β_2 , parameters of target networks: $\bar{\beta}_1 \leftarrow \beta_1, \bar{\beta}_2 \leftarrow \beta_2$.
 - 4: **for** each episode **do**
 - 5: **for** $t \in \{1, \dots, t, \dots, T\}$ **do**
 - 6: **for** $n = 1, 2, \dots, J + K$ **do**
 - 7: Obtain the state observation space using (38) and (39), compute the probability distribution, then select an action from $a(t) \sim \pi_\phi(a(t)|s(t))$.
 - 8: **end for**
 - 9: Calculate every agent’s reward $r(t)$.
 - 10: Obtain optimal $e_n^*(t)$ for each terminal $n \in J$ by solving **SP1**.
 - 11: Allocate UAV resources using **Algorithm 2**.
 - 12: Update queue $Q_j(t), H_{j,k}(t), V_j^Q(t), V_{j,k}^H(t)$ using (9), (10), (24), and (25), and transition to state $s(t+1)$.
 - 13: Store the experience sample $(s(t), a(t), r(t), s(t+1))$ in the experience cache as backup data.
 - 14: Randomly sample experiences from the experience replay pool.
 - 15: **for** $n = 1, 2, \dots, J + K$ **do**
 - 16: Update parameters of Q-function using (44):
 - 17: $\beta_i \leftarrow \beta_i - \lambda \nabla_{\beta_i} J_Q(\beta_i)$, for $i \in \{1, 2\}$.
 - 18: Update parameters of policy-function using (45):
 - 19: $\phi \leftarrow \phi - \lambda \nabla_\phi J_\pi(\phi)$.
 - 20: Update target networks using:
 - 21: $\bar{\beta}_i \leftarrow \tau \beta_i + (1 - \tau) \bar{\beta}_i$, for $i \in \{1, 2\}$, where τ is the learning rate.
 - 22: **end for**
 - 23: **end for**
 - 24: **end for**
-

$\beta, \phi, \bar{\beta}_i$ are updated. Additionally, the Q-function, policy function, and target network parameters of every agent are updated through tracking the learned networks. Moreover, the queue backlog updated continuously throughout the iterations. Once the training phase is complete, the DNN needs only forward propagation, with trained agents using minimal computational resources for decision-making. Additionally, **Algorithm 1** accounts for queue awareness, enabling terminals and UAVs to adjust power and flight trajectories dynamically according to the current task queue status.

C. UAV Resource Allocation

Since **SP3** remains an NP-hard problem, it can not be addressed using traditional convex optimization theory. For such problems, heuristic search algorithms can provide near-optimal solutions. The DOA draws inspiration from the social behavior of Australian dingoes, specifically their hunting

strategies, which include group attacks, harassment, and scavenging. The algorithm also incorporates a survival rate rule for optimizing individuals and is characterized by its strong optimization capabilities and fast convergence [26]. Thus, we apply the DOA to tackle the problem while treating the objective function of **SP3** as the fitness function for the algorithm.

1) *Initial Population Size Generation:*

$$\vec{x}_i(m) = lb_i(m) + rand_i[ub_i(m) - lb_i(m)], \quad (47)$$

where m represents the iteration count. $\vec{x}_i(m)$ denotes the current optimizing individual. lb_i and ub_i represent the lower and upper bounds of $\vec{x}_i$, respectively. $rand_i$ is a random number generated between $[0,1]$.

2) *Three Hunting Strategies:* The dingo exhibits three primary hunting strategies in the wild, each adapted to specific types of prey and environmental conditions. These strategies reflect the high level of cooperation and intelligence the dingoes display in group hunting.

Group attack strategy: When a pack of dingoes hunts large prey, such as kangaroos, they work together to locate the prey and surround it. It is represented as follows

$$\vec{x}_i(m+1) = \beta_1 \sum_{k=1}^{n_a} \left[\frac{(\vec{\varphi}_k(m) - \vec{x}_i(m))}{n_a} - \vec{x}_*(m) \right], \quad (48)$$

where n_a is a randomly generated integer within the range $[2, \frac{S}{2}]$ in descending order. S represents the size set of the optimizing individuals. $\varphi \in X$ and X indicates a randomly generated optimizing individual. $\vec{x}_*(m)$ represents the best solution discovered in the previous iteration, and β_1 is a random number uniformly generated within $[-2, 2]$ that represents the scaling factor, which can alter the dingo's trajectory.

Harassment strategy: When dingoes hunt small animals, such as rabbits, the pack continuously closes in and chases the prey until it is captured. The behavior of the dingo pack in this scenario can be represented as follows

$$\vec{x}_i(m+1) = \vec{x}_*(m) + \beta_1 e^{\beta_2} [\vec{x}_{r_1}(m) - \vec{x}_i(m)], \quad (49)$$

where β_2 is a random number generated between $[-1,1]$. r_1 is a random number within the interval from 1 to the maximum pack size of dingoes. $\vec{x}_{r_1}(m)$ is the randomly selected dingo r_1 in the pack. Besides, $i \neq r_1$.

Scavenging strategy: The scavenging strategy is defined as the behavior of dingoes finding and consuming carrion while wandering randomly in their habitat. This strategy is represented as follows

$$\vec{x}_i(m+1) = \frac{1}{2} e^{\beta_2} [\vec{x}_{r_1}(m) - (-1)^\sigma \vec{x}_i(m)], \quad (50)$$

where σ is a randomly generated binary number, and satisfied $\sigma \in \{0,1\}$.

3) *Optimization Strategy Evaluation:* The use of the group attack strategy, harassment strategy, and scavenging strategy is implemented by comparing two randomly generated probability values (P_1 and P_2) with two predefined probability values

(P and Q). The method for determining which strategy to use is as follows

$$\begin{cases} \text{group attack strategy, } P_1 \geq P \text{ and } P_2 < Q, \\ \text{harassment strategy, } P_1 < P, \\ \text{scavenging strategy, } P_1 \geq P \text{ and } P_2 \geq Q. \end{cases} \quad (51)$$

where P and Q are the probability values for the dingo pack to execute the corresponding strategies, set to fixed values of 0.5 and 0.7, respectively.

4) *Survival Strategy:* The survival rate of dingoes is calculated using the following formula

$$c(i) = \frac{F_{\max} - F(i)}{F_{\max} - F_{\min}}, \quad (52)$$

where $c(i)$ denotes the survival rate of dingo i within the range $[0,1]$. $F_{\max}$ and $F_{\min}$ indicate the maximum and minimum values of the fitness function, with corresponding survival rates of 0 and 1, respectively. $F(i)$ denotes the current fitness function value of the dingo pack i , which is set to the objective value of SP3 in this algorithm.

5) *Dingo Individual Replacement:*

$$\vec{x}_i(m) = \vec{x}_*(m) + \frac{\vec{x}_{r_1}(m) - (-1)^{-\sigma} \vec{x}_{r_2}(m)}{2}, \quad (53)$$

where $\vec{x}_{r_1}(m)$ and $\vec{x}_{r_2}(m)$ are individuals randomly selected from the dingo population $\vec{x}_i(m)$. r_1 and r_2 are random numbers generated within the interval from 1 to the maximum dingo population size, with $r_1 \neq r_2$.

The specific algorithm is shown in **Algorithm 2**. Firstly, initialize the dingo population and evaluate the fitness for each individual. In each time slot, update the positions of the dingoes through multiple iterations. If the random numbers meet certain conditions, different strategies (such as group attack, harassment, or scavenging) are used to update positions, and the survival rate of the dingoes is calculated. Individuals with low survival rates are replaced. Through these steps, the global optimal individual positions are ultimately found, achieving optimization of computational resources.

D. Complexity Analysis

SP1 is a convex subproblem, optimized through I_1 iterations with a complexity of $O(I_1 K)$. For the MASAC algorithm, its major complexity stems from the offline training, and the online implementation with well-trained actor networks consumes negligible time. Define L and I as the number of layers in the actor network and the critic network, respectively. Let L_l and I_i represent the number of neurons in the l layer and the i layer, respectively. Then, the training complexity is given by $O\left(ep_{\max} T(J+K) \left(\sum_{l=2}^{L-1} J_{l-1} (L_l + L_l L_{l+1}) + \sum_{i=2}^{I-1} I_{i-1} (I_i + I_i I_{i+1})\right)\right)$, where $ep_{\max}$ is the number of episodes, and T is the total number of time slots. Such complexity increases linearly with the number of terminals and UAVs. For CRO, the total computational complexity across all UAVs is $O(I_{DOA} N_0 JK)$, where N_0 is the number of dingoes. Compared to PSO, which has a complexity of $O(I_{PSO} N_{PSO} JK)$, CRO would introduce additional computational overhead due to its dynamic survival rate

Algorithm 2 DOA-Based Computing Resource Optimization Algorithm (CRO)

```

1: Input: Dingo population size.
2: Output: Global optimal individual position.
3: Initialize: Iterations  $I_{DOA}$ , time  $T$ , and other parameters.
4: Initialize the dingo population and calculate the fitness of
   individuals.
5: Set parameters: population size,  $P$ ,  $Q$ .
6: for  $t = 1$  to  $T$  do
7:   for  $Iteration = 1$  to  $Iterations$  do
8:     for  $k = 1, 2, \dots, K$  do
9:       if random number  $P_1 < P$  then
10:        if random number  $P_2 < Q$  then
11:          Update the positions of dingoes using the
            group attack strategy (48).
12:        else
13:          Update the positions of dingoes using the
            harassment strategy (49).
14:        end if
15:        else
16:          Update the positions of dingoes using the
            scavenging strategy (50).
17:        end if
18:        Calculate the survival rate of the dingo individ-
            uals using (52).
19:        Perform population replacement for individuals
            with low survival rates using (53).
20:        if  $x_{new} < x_*$  then
21:           $x_* = x_{new}$ .
22:        end if
23:         $Iteration = Iteration + 1$ .
24:      end for
25:    end for
26:  end for

```

mechanism and behavioral strategies like group attacks and harassment. However, this added complexity enables CRO to achieve better global exploration and local exploitation, particularly in high-dimensional and dynamic optimization scenarios. As a result, the total computational complexity is $O\left(I_1 K + ep_{\max} T(J + K) \left(\sum_{l=2}^{L-1} (I_{l-1} L_l + L_l L_{l+1}) + \sum_{i=2}^{I-1} I_{i-1} (I_i + I_i I_{i+1})\right) + I_{DOA} J N_0 K\right)$.

VI. SIMULATION RESULTS

This section evaluates the proposed method through simulations. Consider a 1000 m $\times$ 1000 m area with 3 UAVs and 45 devices distributed within it. The UAVs fly at a height of $H = 100$ m and a speed of $V = 20$ m/s. Furthermore, we configure the transaction size as 2 KB and the block size as 2 MB, respectively. The other parameters are detailed in Table III[1], [5], [16].

We evaluate our algorithm against the following benchmark methods:

- 1) *Trajectory planning and Resource allocation MASAC without trajectory planning (NT-MASAC)* [27]: Resource allocation for both the terminals and UAVs has been

TABLE III
SIMULATION PARAMETERS

Parameter	Value
Total service time T	200 s
Length of time slot τ	0.5 s
Number of UAVs	3
Altitude of UAV	100 m
Safe distance between two UAVs	5 m
Additive loss $\eta_{j,k}^{\text{LoS}}, \eta_{j,k}^{\text{NLoS}}$	0.1, 2.1
Carrier frequency f_c	0.1 GHz
Environment parameters ρ_1, ρ_2	4.88, 0.43
CPU frequency of each UAV	10 GHz
Number of terminals	45
Maximum transmission power of terminals	23 dBm
Bandwidth	1 MHz
Queuing delay bound $\delta_{j,\max}^Q, \delta_{j,k,\max}^H$	100 ms, 50 ms
Duration of a time slot T	0.5 s
Noise power	-144 dBm/Hz
Reference channel gain h_0	-50 dB
Processing density for block in each slot, $L_k(t)$	737 cycles/bit
Weight parameter λ	10^{11}

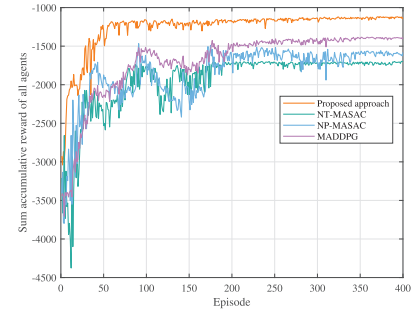


Fig. 3. Sum accumulative reward over episode.

optimized. However, the UAV's flight trajectory is assumed to follow a series of concentric circles for simplicity.

- 2) *Trajectory planning and Resource allocation MASAC without power optimization (NP-MASAC)* [28]: We leverage MASAC to train the trajectory planning of UAVs, while omitting the terminal power control.
- 3) *MADDPG* [4]: Terminal data sensing rate and UAV resource allocation are optimized using the proposed method. Meanwhile, terminal transmission power and UAV trajectories are addressed using MADDPG.
- 4) *PSO Algorithm (PSO)* [29]: Data sensing rate, transmission power, and UAV trajectories are optimized using the proposed method, while UAV resource allocation is addressed using particle swarm optimization.

Fig. 3 illustrates the sum accumulative reward throughout the training episode. The figure indicates that our algorithm performs better and converges faster. This is because MASAC encourages exploration by maximizing policy entropy, enhancing policy robustness and adaptability, and thus outperforms MADDPG and other reduction strategy methods in complex multi-agent environments. Additionally, the curve exhibits higher volatility at the start of training phase, which is attributed to terminals learning power control to adapt to dynamic environments such as task arrivals, noise interference, and UAV movements, while UAV explores flight trajectory for serving multiple terminals as well as avoids violating boundary and safety distance constraints. Once all agents learn the appropriate actions, the proposed method converges, as

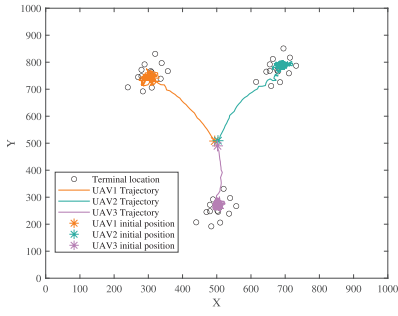


Fig. 4. Top view of UAV trajectories.

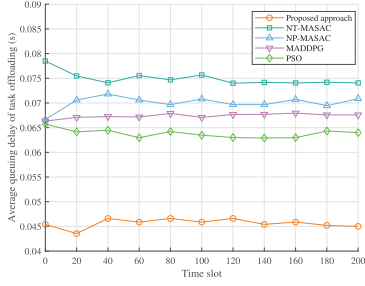
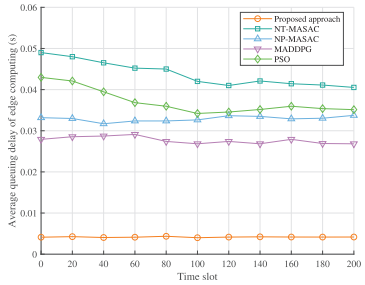
(a) Queuing delay of $Q_j(t)$.(b) Queuing delay of $H_{j,k}(t)$.

Fig. 5. Average queuing delay over time.

observed when the episode approaches 100. However, due to the dynamic changes in the network environment, the cumulative reward may fluctuate around its convergent value.

Fig. 4 depicts the UAV trajectories in a scenario with 3 UAVs and 45 devices. In practice, for IoT data collection or sensing, terminals are generally deployed near specific targets, so we distribute the terminals across several hotspot areas. As shown in the figure, each UAV can move from its initial position to the associated terminals and then hover near the target area. This is because the UAVs adjust their trajectories based on the dynamic task offloading demands of the terminals they serve, ensuring effective computation offloading services and significantly reducing overall system overhead.

Fig. 5 (a)-(b) presents the average queuing delays of $Q_j(t)$ and $H_{j,k}(t)$ over time, where each point represents a moving window average over 20 time slots. It can be observed that all methods provide relatively stable data queues. However, the proposed method ensures lower queue delays and better stability. This is because the proposed method controls queue data admission and computation more effectively through the joint optimization of data sensing rates and computational resources, maintaining the stability of the queues. According to the numerical results, compared to NT-MASAC, NP-MASAC,

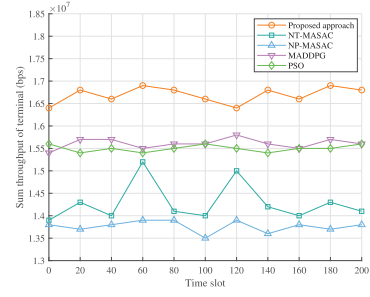


Fig. 6. Sum throughput of terminal over time.

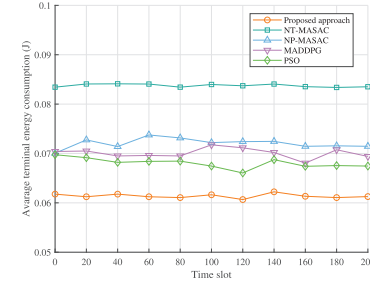


Fig. 7. Average terminal energy consumption.

MADDPG, and PSO, the proposed method can reduce queue delay by 30.77%, 35.71%, 30.73%, and 29.47%, respectively.

Fig. 6 compares the system throughput of our method with that of the benchmark algorithms, where all algorithms except NP-MASAC perform power control. The numerical results indicate that our method consistently maintains a high and stable level of throughput. It can also be observed from the figure that power control enables better utilization of channel capacity for task offloading. Furthermore, different optimization dimensions and methods result in varying UAV trajectories, leading to different system throughputs across the algorithms. Additionally, due to the fixed circular UAV trajectory, the throughput of NT-MASAC exhibits periodic fluctuations.

Fig. 7 shows average energy consumption of terminals with the maximum perceived data volume of $S_{\max} = 2 \times 10^5$ bit. The results indicate that the proposed approach delivers a lower average system energy consumption than the four benchmark methods. On one hand, NP-MASAC does not perform power optimization, resulting in the terminals always offloading tasks at maximum transmission power, resulting in increased energy consumption. On the other hand, although other methods include power control, different optimization dimensions result in varying resource allocations and UAV trajectories, leading to differences in average energy consumption among the terminals. Additionally, NT-MASAC exhibits significant fluctuations in average energy consumption due to its periodic changes in throughput.

Fig. 8 depicts how the maximum available computational resources of the UAVs affect the total queuing delay and block creation delay. As the computational capacity $f_{k,\max}$ rises from 10 GHz to 30 GHz, the total queuing delay initially decreases before stabilizing. While the computational capacity rises from 10 GHz to 25 GHz, the total queuing delay and block creation delay decrease by 50.47% and 46.75%, respectively. The figure also reveals that, further increases in

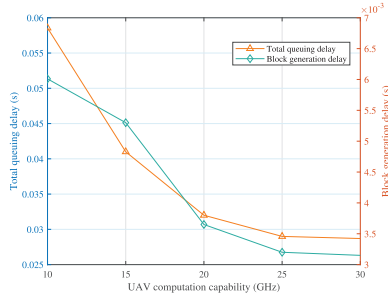


Fig. 8. The effects of UAV maximum computation capability on delay and security performance.

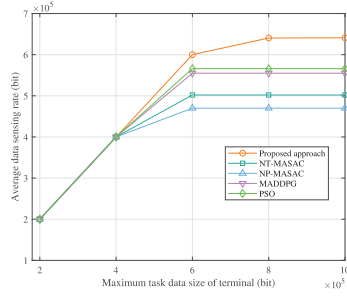


Fig. 9. Average data sensing rate versus the maximum task bits of terminal.

computational capacity do not result in significant reductions in delay when $f_{k,\max}$ exceeds 25 GHz. This is because, with sufficient computational resources, transmission rate becomes the new limiting factor in decreasing the total queuing delay.

In Fig. 9, we further evaluate the system's average data sensing rate under different task data scales. The average sensing rates for all methods increase with the maximum sensing data amount $S_{\max}$ and eventually stabilize. On one hand, due to limited data processing resources, the amount of admitted data gradually stabilizes to maintain queue stability and prevent excessive queue buildup. On the other hand, with limited transmission power at the terminals, the amount of tasks sent per time slot also stabilizes once the queue buildup reaches a certain level. The proposed algorithm consistently outperforms other methods because it jointly optimizes task sensing, transmission power, computing resources, and UAV trajectories, thus maximizing the average sensing rate while controlling transmission and computation queue buildup. Numerical results show that when the task data scale is 8×10^5 bit, the proposed method improves the sensing rate by 27.59%, 36.27%, 15.41%, and 13.16% compared to the NT-MASAC, NP-MASAC, MADDPG, and PSO algorithms, respectively.

Fig. 10 shows the system's average data sensing rate as the number of terminal devices increases. It can be observed that the average sensing rate decreases with the increase in the number of terminals. The decline is due to the increased interference among terminal devices, which affects the overall system throughput. Additionally, As the computational resources at the edge nodes approach saturation, the computation speed slows down, further exacerbating task queue backlogs. However, the proposed method consistently achieves the highest average sensing rate. This is attributed to the joint optimization of task sensing, transmission power, UAV computational resources, and trajectory, which boosts resource efficiency and ensures scalability. In contrast, other algorithms,

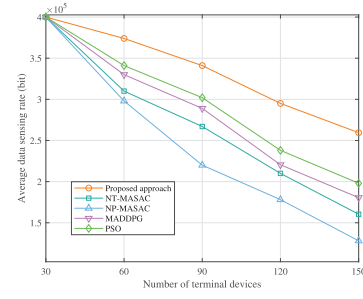


Fig. 10. Average data sensing rate versus the numbers of terminals.

lacking effective dynamic network adjustment mechanisms, struggle to address the dual challenges of queue backlogs and offloading efficiency in large-scale scenarios.

VII. CONCLUSION

This paper proposes a UAV-enabled secure aerial computing network architecture which integrates MEC and blockchain technologies to support computation offloading for remote IoT terminals. Under this architecture, task data sensed by terminals is offloaded to UAVs for edge processing. We address the problem of maximizing the long-term average data sensing rate of terminals by jointly optimizing terminal data sensing, transmission power, UAV trajectory planning, along with UAV resource allocation while adhering to long-term queue delay and security constraints. Specifically, we design an AGIN-MASAC based on MADRL to collaboratively train terminal and UAV agents. Additionally, by utilizing convex optimization and the Dingo algorithm, we embed two subroutines within AGIN-MASAC to optimize data sensing rates and UAV resource allocation. Numerical results show that our method outperforms other benchmark methods in terms of long-term average data sensing rate, training convergence, and queue delay.

REFERENCES

- [1] P. Qin et al., "Optimal resource allocation for AGIN 6G: A learning-based three-sided matching approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 1553–1565, Apr. 2024.
- [2] Y. Chen, K. Li, Y. Wu, J. Huang, and L. Zhao, "Energy efficient task offloading and resource allocation in air-ground integrated MEC systems: A distributed online approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 8, pp. 8129–8142, Aug. 2024.
- [3] W. Zhang, G. Zhang, and S. Mao, "Deep-reinforcement-learning-based joint caching and resources allocation for cooperative MEC," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 12203–12215, Apr. 2024.
- [4] P. Qin, Y. Fu, Y. Xie, K. Wu, X. Zhang, and X. Zhao, "Multi-agent learning-based optimal task offloading and UAV trajectory planning for AGIN-power IoT," *IEEE Trans. Commun.*, vol. 71, no. 7, pp. 4005–4017, Jul. 2023.
- [5] D. Wang, Y. Jia, M. Dong, K. Ota, and L. Liang, "Blockchain-integrated UAV-assisted mobile edge computing: Trajectory planning and resource allocation," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 1263–1275, Jan. 2024.
- [6] J. Wang, Z. Jiao, J. Chen, X. Hou, T. Yang, and D. Lan, "Blockchain-aided secure access control for UAV computing networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5267–5279, Dec. 2024.
- [7] P. Qin, Y. Fu, J. Zhang, S. Geng, J. Liu, and X. Zhao, "DRL-based resource allocation and trajectory planning for NOMA-enabled multi-UAV collaborative caching 6G network," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 8750–8764, Jun. 2024.
- [8] D. Wu, T. Liu, Z. Li, T. Tang, and R. Wang, "Delay-aware edge-terminal collaboration in green Internet of Vehicles: A multi-agent soft actor-critic approach," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 1090–1102, Jun. 2023.

- [9] H. Feng, J. Wang, Z. Fang, J. Chen, and D.-T. Do, "Evaluating AoI-centric HARQ protocols for UAV networks," *IEEE Trans. Commun.*, vol. 72, no. 1, pp. 288–301, Jan. 2024.
- [10] Z. Yang, S. Bi, and Y. A. Zhang, "Online trajectory and resource optimization for stochastic UAV-enabled MEC systems," *IEEE Trans. Wireless Commun.*, vol. 21, no. 7, pp. 5629–5643, Jul. 2022.
- [11] N. Lin, H. Tang, L. Zhao, S. Wan, A. Hawbani, and M. Guizani, "A PDDQNLP algorithm for energy efficient computation offloading in UAV-assisted MEC," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8876–8890, Dec. 2023.
- [12] P. Qin, Y. Fu, X. Zhao, K. Wu, J. Liu, and M. Wang, "Optimal task offloading and resource allocation for C-NOMA heterogeneous air-ground integrated power Internet of Things networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9276–9292, Nov. 2022.
- [13] X. Fan, H. Zhou, K. Sun, X. Chen, and N. Wang, "Channel assignment and power allocation utilizing NOMA in long-distance UAV wireless communication," *IEEE Trans. Veh. Technol.*, vol. 72, no. 10, pp. 12970–12982, Oct. 2023.
- [14] X. Li, S. Bi, Z. Quan, and H. Wang, "Online cognitive data sensing and processing optimization in energy-harvesting edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 6611–6626, Aug. 2022.
- [15] J. Shi, J. Du, Y. Shen, J. Wang, J. Yuan, and Z. Han, "DRL-based V2V computation offloading for blockchain-enabled vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3882–3897, Jul. 2023.
- [16] Q. Tang, Z. Fei, J. Zheng, B. Li, L. Guo, and J. Wang, "Secure aerial computing: Convergence of mobile edge computing and blockchain for UAV networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 11, pp. 12073–12087, Nov. 2022.
- [17] X. Ye, M. Li, P. Si, R. Yang, Z. Wang, and Y. Zhang, "Collaborative and intelligent resource optimization for computing and caching in IoV with blockchain and MEC using A3C approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 1449–1463, Feb. 2023.
- [18] A. M. Seid, J. Lu, H. N. Abishu, and T. A. Ayall, "Blockchain-enabled task offloading with energy harvesting in multi-UAV-assisted IoT networks: A multi-agent DRL approach," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 12, pp. 3517–3532, Dec. 2022.
- [19] D. Wu et al., "UAV-assisted real-time video transmission for vehicles: A soft actor-critic DRL approach," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14710–14726, Apr. 2024.
- [20] A. Gao, S. Zhang, Y. Hu, W. Liang, and S. X. Ng, "Game-combined multi-agent DRL for tasks offloading in wireless powered MEC networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9131–9144, Jul. 2023.
- [21] A. M. Seid, A. Erbad, H. N. Abishu, A. Albaser, M. Abdallah, and M. Guizani, "Blockchain-empowered resource allocation in multi-UAV-enabled 5G-RAN: A multi-agent deep reinforcement learning approach," *IEEE Trans. Cogn. Commun. Netw.*, vol. 9, no. 4, pp. 991–1011, Aug. 2023.
- [22] H. Wang, H. Zhang, X. Liu, K. Long, and A. Nallanathan, "Joint UAV placement optimization, resource allocation, and computation offloading for THz band: A DRL approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 7, pp. 4890–4900, Jul. 2023.
- [23] Z. Chang, H. Deng, L. You, G. Min, S. Garg, and G. Kaddoum, "Trajectory design and resource allocation for multi-UAV networks: Deep reinforcement learning approaches," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2940–2951, Oct. 2023.
- [24] P. Qin et al., "Joint resource allocation and UAV trajectory design for D2D-assisted energy-efficient air-ground integrated caching network," *IEEE Trans. Veh. Technol.*, vol. 73, no. 11, pp. 17558–17571, Nov. 2024.
- [25] J. Wang, J. Wang, Z. Tong, Z. Jiao, M. Zhang, and C. Jiang, "ACBFT: Adaptive chained Byzantine fault-tolerant consensus protocol for UAV ad hoc networks," *IEEE Trans. Veh. Technol.*, early access, Mar. 25, 2025, doi: [10.1109/TVT.2025.3548281](https://doi.org/10.1109/TVT.2025.3548281).
- [26] J. Li et al., "Multidomain separation for human vital signs detection with FMCW radar in interference environment," *IEEE Trans. Microw. Theory Techn.*, vol. 72, no. 7, pp. 4278–4293, Jul. 2024.
- [27] Y. Wang, H. Wu, R. H. Jhaveri, and Y. Djenouri, "DRL-based URLLC-constraint and energy-efficient task offloading for Internet of Health Things," *IEEE J. Biomed. Health Informat.*, vol. 28, no. 6, pp. 3305–3316, Jun. 2024.
- [28] X. Li, Y. Qin, J. Huo, and W. Huangfu, "Computation offloading and trajectory planning of multi-UAV-enabled MEC: A knowledge-assisted multiagent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 7077–7088, May 2024.

- [29] L. Sun, L. Wan, and X. Wang, "Learning-based resource allocation strategy for industrial IoT in UAV-enabled MEC systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 5031–5040, Jul. 2021.



Peng Qin (Member, IEEE) received the B.S. and Ph.D. degrees in information and communication engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2009 and 2014, respectively. From 2012 to 2013, he was a Visiting Scholar with the University of Victoria, Victoria, BC, Canada. He is currently an Associate Professor with the School of Electrical and Electronic Engineering, North China Electric Power University. His research interests include resource allocation in the Internet of Things, smart grid communications, space air ground integrated networks, and vehicular networks. He was a recipient of the International Communications Signal Processing and Systems Conference Best Paper Award and the International Conference on Artificial Intelligence in China Best Paper Award in 2019, 2020, 2021, and 2022, respectively.



Min Fu received the B.S. degree from North China Electric Power University in 2022, where she is currently pursuing the M.Sc. degree with the School of Electrical and Electronic Engineering. Her research interests include space air ground integrated networks and wireless communication network resource optimization.



Yang Fu received the B.S. degree from North China Electric Power University in 2022, where he is currently pursuing the Ph.D. degree with the School of Electrical and Electronic Engineering. His research interests include resource allocation in smart grid communications, space air ground integrated networks, vehicular networks, and the Internet of Things.



Jingjing Wang (Senior Member, IEEE) received the B.Sc. degree (Hons.) in electronic information engineering from Dalian University of Technology, Liaoning, China, in 2014, and the Ph.D. degree (Hons.) in information and communication engineering from Tsinghua University, Beijing, China, in 2019. From 2017 to 2018, he visited the Next Generation Wireless Group, University of Southampton, U.K., chaired by Prof. Lajos Hanzo. He is currently a Professor at the School of Cyber Science and Technology, Beihang University, Beijing, and also a Researcher at Hangzhou Innovation Institute, Beihang University, Hangzhou, China. He has published more than 100 IEEE journal/conference papers. His research interests include AI enhanced next-generation wireless networks, UAV networking, and swarm intelligence. He was a recipient of the Best Journal Paper Award of the IEEE ComSoc Technical Committee on Green Communications and Computing and the Best Paper Award of the IEEE ICC and the IEEE IWCMC. He is currently serving as an Editor for IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE INTERNET OF THINGS JOURNAL, and IEEE WIRELESS COMMUNICATIONS LETTER.